Sparse Matrices: 5 Tips and Tricks

12 Sep 2018

3 minute read

Over the course of my internship at the online shopping company Wish, I have dealt a lot with a lot of data in the form of sparse matrices, specificaly in the form of item interaction matrices for customer data. In doing so, I have made heavy use of scipy's sparse matrices library. Here are 5 tricks that I have learned.

Note this post has an associated Jupyter notebook that contains example code.

1: Use a normal dict instead of a `dok_matrix` to construct sparse matrices incrementally

A dok matrix is essentially storing a sparse matrix in a hashmap. According to the documentation, it is an efficient structure for constructing matrices incrementally, because adding an element to a hashmap is an $O(1)$ operation. However, what the documentation doesn't mention is that the `dok_matrix` class has a very significant overhead on item assignment. Suppose you have a dok matrix and you try to perform item assignment via:

```
matrix[i, j] = some_value
```

This will end up calling the object's `__setitem__` method, whose implementation can be seen here. Looking at the code for this method, there is a ton of type checking on the arguments, which is really inefficient if you know that your arguments are of the right type.

A faster way to do it is to use an ordinary default dict, then directly update the underlying dictionary of the dok class to construct a dok matrix.

If `matrix_dict` is a default dict with (i, j) tuples as keys, and `matrix` is a dok matrix, then the key line to do the update is:

```
dict.update(matrix, matrix_dict)
```

See this post's notebook for more detail. I noticed a 20x speedup using this trick.

2: When using compressed sparse row (csr) format, it is faster to convert the whole matrix to compressed sparse column format and back than do a bunch of column selections

When dealing with a large co-purchase matrix, I came across this problem of having to zero out a bunch of columns to account for items that were missing in the next steps of my data analysis (I was trying to use the matrix to calculate a conditional distribution of item co-purchases). But with the csr format, changing this takes an incredibly long time! I found it was faster to convert to csc and back.

However, it isn't always faster, as shown in my jupyter notebook. But is is always an option to consider. I've only seen it be faster in very sparse matrices (density about 1e-6), where I had to change about 10% of columns. For most use cases, it tends to be slower.

3: Don't be afraid to access the internal structure of csr/csc matrices

Calling `getrow()` will make a copy of the data, which is obviously slower (see code) It is much faster to access the underlying arrays yourself. See the notebook for an example.

4: When doing adding operations, convert to csr format, since scipy does it internally anyways

Looking at the base file for sparse matrices, a lot of the implementations of basic functions (e.g. min, multiply, equality checking) work by first converting to a csr matrix. So if you are doing these operations you might as well be in csr format to begin with.

5: NEVER convert from CSR to DOK format with a large matrix- use COO instead

Converting to a COO format is fast, and consists of just a few array swaps (see source code). Converting to a dok matrix on the other hand is *extremely* inefficient, mainly because it *converts to a COO matrix anyways as an intermediate step*! (source).

Experimentally, I found converting to a COO matrix to be about 100x faster than converting to DOK with a large number of samples, and about twice as fast with a small number of samples.

Github: github.com/AustinT
Professional Email: austin period james period tripp AT gmail period com
Cambridge Email: [first/middle/last initial]212 AT cam.ac.uk
Google Scholar

Site last updated: 2022-07-14.